



Day 10: Methods, Business Logic & Fund Transfer in Java

1. What does a Software Engineer actually do?

Software Engineering is not just about writing code; it involves the complete lifecycle:

- **Development:** Writing new features and building applications.
- **Maintenance:**
 - **Enhancements:** Adding new capabilities over time.
 - **Bug Fixing:** Resolving defects and edge cases.
 - **Monitoring & Health Checks:** Ensuring application uptime, speed, and reliability.

Real-World Analogy:

Buying a car is like initial **Development/Sale**. Taking it to the service center for regular servicing, oil changes, or fixing issues is **Maintenance**.

2. Business Logic & Methods (Functions)

What is Business Logic?

The actual rule or calculation behind a feature (e.g., adding two numbers, transferring money, authenticating a user).

What is a Method?

A method represents a **Task/Action**. It wraps data and business logic together.

4 Pillars of a Method Signature:

1. **Method Name:** Identifies the action (e.g., pay, doTransaction). Follow camelCase.
2. **Parameters (Input):** What data the method needs to perform the task (e.g., int amount, String senderAccNo).
3. **Method Body { ... }:** The actual execution logic / business logic.
4. **Return Type (Output):** The result produced after execution (e.g., boolean for success/failure, int for sums).

3. Types of Methods in Java

1. **Built-in Methods:** Pre-defined by Java / Oracle library (e.g., System.out.println(), Integer.parseInt()).
2. **User-Defined Methods:** Custom methods written by developers based on project requirements.

4. Practical Example: Fund Transfer Logic

Requirement:

Transfer money from sender to receiver.

- **Input:** Amount to transfer, Sender Account, Receiver Account.
- **Output:** true (success) or false (failure).

Math Behind the Balance:

- Sender starts with: 100
- Receiver starts with: 200
- Transfer amount: 10
- **Calculation:**

$$\text{senderBalance} = \text{senderBalance} - \text{txnamount} \implies 100 - 10 = 90$$

$$\text{recBalance} = \text{recBalance} + \text{txnamount} \implies 200 + 10 = 210$$

Java Implementation:

```
public class FundTransfer {

    // User-defined method to handle fund transfer
    public static boolean doTransaction(int amountToTransfer, String senderAccNo, String
recAccNo) {
        System.out.println("Processing transaction of ₹" + amountToTransfer);
        System.out.println("From: " + senderAccNo + " -> To: " + recAccNo);

        // Simulated business logic
        // In real systems: check balance, deduct, add, verify
        boolean isSuccess = true;

        return isSuccess;
    }

    public static void main(String[] args) {
        System.out.println("Transaction initiated...");

        // Calling the method with arguments
        boolean result = doTransaction(10, "987654321", "123456789");

        System.out.println("Is transaction successful? " + result);
    }
}
```

5. Architectural Principle: SOLID (Letter "S")

- **Single Responsibility Principle (SRP):**
 - A method or class should have **one, and only one reason to change**.
 - Keep your methods focused: if a method transfers funds, it shouldn't also send marketing emails or format UI strings.